

# INF F5 — Programmation avancée et structures de données

## Entraînement — Gestion des notes d'une promotion

*L3 MIASHS — Notes de cours, TD et TM autorisés — durée conseillée : 2 h 30*

*Le barème indiqué est indicatif. Ce barème est sur 24 points, la note finale sera le minimum entre la note obtenue selon ce barème et 20. Lorsque des définitions de la même méthode sont demandées sur plusieurs classes, rappeler le nom de la classe concernée en commentaire.*

## Sujet de programmation : Gestion des notes d'une promotion

Nous nous intéressons à la gestion des notes des étudiants d'une promotion. Chaque étudiant possède une liste de notes comprises entre 0 et 20, à partir de laquelle on calcule sa moyenne. Une promotion regroupe des étudiants que l'on veut classer, filtrer et, pour les moins bons, convoquer à une session de rattrapage. Les notes sont attribuées par des enseignants.

Par exemple, considérons la promotion suivante, inscrite dans cet ordre :

- Alice MARTIN, numéro E001, notes [12.0, 15.5, 9.0], moyenne 12.17
- Bob PETIT, numéro E002, notes [8.0, 11.0], moyenne 9.50
- Chloé DURAND, numéro E003, notes [14.0, 16.0, 18.0], moyenne 16.00
- Dan LEROY, numéro E004, aucune note
- Eva ROUX, numéro E005, notes [5.0, 7.0], moyenne 6.00

Le classement (étudiants ayant au moins une note, par moyenne décroissante) est : Chloé, Alice, Bob, Eva. Dan n'y figure pas car il n'a aucune note. Les admis (moyenne supérieure ou égale à 10, dans l'ordre d'inscription) sont Alice et Chloé. La file de rattrapage (moyenne strictement inférieure à 10, du plus faible au meilleur) contient Eva puis Bob.

La hiérarchie de classes est la suivante :

```
Personne (abstraite)
```

```
├─ Etudiant
└─ Enseignant
```

```
Promotion<E extends Etudiant>
```

```
├─ etudiants    : List<E>          (ordre d'inscription)
└─ rattrapage   : LinkedList<E>    (file de rattrapage)
```

La séquence d'instructions suivante construit la promotion de l'exemple (les blocs try/catch sont omis) :

```
Promotion<Etudiant> promo = new Promotion<>();
Etudiant alice = new Etudiant("Martin", "Alice", "E001");
Etudiant bob   = new Etudiant("Petit", "Bob", "E002");
Etudiant chloe = new Etudiant("Durand", "Chloé", "E003");
promo.inscrire(alice);      // true
promo.inscrire(bob);       // true
promo.inscrire(chloe);     // true
promo.inscrire(alice);     // false : déjà inscrit

Enseignant moreau = new Enseignant("Moreau", "Julie", "Statistiques");
moreau.noter(alice, 12.0);
moreau.noter(alice, 15.5);
moreau.noter(alice, 9.0);
moreau.noter(bob, 8.0);
moreau.noter(bob, 21.0);    // lève NoteInvalideException
// ... et ainsi de suite pour les autres notes
```

## 1 Classe Personne (2 points)

Une instance de la classe Personne concentre l'identité commune aux étudiants et aux enseignants. On mémorise dans l'attribut nom le nom de famille (toujours en majuscules) et dans l'attribut prenom le prénom. La classe est abstraite : le rôle d'une personne est précisé par ses sous-classes grâce à la méthode `getRole()`.

**1.1 (1 pt)** Donner la définition du constructeur de cette classe. Si nom ou prenom est null, ou ne contient que des espaces, une `IllegalArgumentException` doit être levée. Le nom est mémorisé en majuscules.

**1.2 (0,5 pt)** Donner la définition des accesseurs en lecture `getNom()` et `getPrenom()`.

**1.3 (0,5 pt)** Donner la définition de la méthode `toString()` qui retourne le prénom, suivi d'un espace, suivi du nom. Par exemple "Alice MARTIN".

## 2 Classe Etudiant (6 points)

La classe Etudiant est une sous-classe de Personne. Un étudiant est identifié par son numéro (attribut numero, qui ne change jamais) et possède une liste de notes (attribut notes). Cette liste est vide à la création de l'étudiant. Elle est déclarée avec le type `List<Double>` et instanciée avec une `ArrayList`.

**2.1 (1 pt)** Donner la définition du constructeur de cette classe. Si numero est null ou vide, une `IllegalArgumentException` doit être levée. Le constructeur de `Personne` doit être appelé.

**2.2 (1 pt)** Donner la définition des accesseurs en lecture `getRole()`, qui retourne "Étudiant", `getNumero()` et `getNotes()`. La méthode `getNotes()` ne doit pas permettre à l'appelant de modifier la liste interne de l'étudiant : expliquer en commentaire la technique employée.

**2.3 (1,5 pt)** Donner la définition de la méthode `ajouterNote(double note)` qui ajoute la note à la fin de la liste. Elle lève une `NoteInvalideException` si la note n'est pas comprise entre 0 et 20 (bornes incluses) ou si elle n'est pas un nombre (`Double.isNaN()`). En cas d'exception, l'étudiant ne doit pas avoir été modifié.

**2.4 (1,5 pt)** Donner la définition de la méthode `moyenne()` qui retourne la moyenne arithmétique des notes de l'étudiant, ou lève une `IllegalStateException` s'il n'a aucune note. Expliquer en commentaire pourquoi retourner 0 dans ce cas serait une mauvaise idée.

**2.5 (1 pt)** Donner les définitions des méthodes `equals(Object)` et `hashCode()`. Deux étudiants sont égaux si et seulement si ils ont le même numéro. Le contrat entre `equals` et `hashCode` doit être respecté, et `equals` doit gérer correctement null et les objets d'une autre classe.

### 3 Classe Enseignant (2 points)

La classe `Enseignant` est une sous-classe de `Personne`. On mémorise dans l'attribut `matiere` la matière enseignée.

**3.1 (0,5 pt)** Donner la définition du constructeur de cette classe. Si `matiere` est null ou vide, une `IllegalArgumentException` doit être levée.

**3.2 (0,5 pt)** Donner la définition des accesseurs en lecture `getRole()`, qui retourne "Enseignant", et `getMatiere()`.

**3.3 (1 pt)** Donner la définition de la méthode `noter(Etudiant e, double note)` qui attribue la note à l'étudiant `e` en utilisant sa méthode `ajouterNote`. Si `e` est null, une `IllegalArgumentException` doit être levée. L'exception `NoteInvalideException` ne doit pas être attrapée dans cette méthode : expliquer en commentaire pourquoi.

## 4 Classe Promotion (6 points)

La classe générique `Promotion<E extends Etudiant>` représente un ensemble d'étudiants. L'attribut `etudiants` référence la liste des étudiants inscrits, dans l'ordre d'inscription (instanciée avec une `ArrayList`). L'attribut `rattrapage`, utilisé à la section 6, référence une file d'étudiants (`LinkedList`).

**4.1 (0,5 pt)** Donner la définition du constructeur sans paramètre de cette classe : les deux listes sont créées vides.

**4.2 (1,5 pt)** Donner la définition de la méthode `inscrire(E e)` qui ajoute `e` à la fin de la liste des étudiants inscrits. Si `e` est null, une `IllegalArgumentException` doit être levée. Cette méthode retourne `true` si et seulement si `e` a bien été ajouté : elle retourne donc `false` lorsqu'un étudiant égal à `e` est déjà inscrit.

**4.3 (1,5 pt)** Donner la définition de la méthode `trouver(String numero)` qui retourne l'étudiant inscrit dont le numéro est égal à `numero`, et lève une `EtudiantIntrouvableException` sinon. Si `numero` est null, une `IllegalArgumentException` doit être levée.

**4.4 (1 pt)** Donner la définition de la méthode `admis()` qui retourne une nouvelle liste contenant les étudiants ayant au moins une note et dont la moyenne est supérieure ou égale à 10, dans l'ordre d'inscription.

**4.5 (1,5 pt)** Donner la définition de la méthode `classement()` qui retourne une nouvelle liste contenant les étudiants ayant au moins une note, triés par moyenne décroissante. À moyenne égale, l'ordre d'inscription est conservé. La liste des étudiants inscrits ne doit pas être modifiée par cette méthode.

## 5 Synthèse du classement (2 points)

**5.1 (0,5 pt)** Donner la définition de la méthode `toString()` dans la classe `Etudiant` qui retourne une `String` de la forme `"(E003:Chloé DURAND:3:16.00)"`, soit le numéro de l'étudiant, suivi de son prénom et de son nom, suivi de son nombre de notes, suivi de sa moyenne avec deux décimales et un point comme séparateur décimal (`String.format(Locale.US, "%.2f", ...)`). Lorsque l'étudiant n'a aucune note, la moyenne est remplacée par un tiret : `"(E004:Dan LEROY:0:-)"`.

**5.2 (1,5 pt)** Donner la définition de la méthode `synthese()` dans la classe `Promotion`. Cette méthode retourne `"[]"` lorsque le classement est vide, et un résultat similaire à `"[(E003:Chloé DURAND:3:16.00), (E001:Alice MARTIN:3:12.17), (E002:Bob PETIT:2:9.50), (E005:Eva ROUX:2:6.00)]"` sinon. Elle doit s'appuyer sur la méthode `classement()`.

## 6 File de rattrapage (3 points)

Les étudiants ayant au moins une note et dont la moyenne est strictement inférieure à 10 sont convoqués à une session de rattrapage, un par un, du plus faible au meilleur. Ils sont mémorisés dans l'attribut `rattrapage` de la classe `Promotion`, de type `LinkedList<E>`.

**6.1 (1,5 pt)** Donner la définition de la méthode `constituerRattrapage()` qui vide la file puis la remplit avec les étudiants concernés, du plus faible au meilleur. Justifier en commentaire le choix de `LinkedList` plutôt que `ArrayList` pour cet attribut.

**6.2 (1,5 pt)** Donner la définition de la méthode `convoquerSuivant()` qui retire de la file l'étudiant situé en tête et le retourne, ou lève une `FileVideException` si la file est vide. Sur l'exemple de la section 1, après `constituerRattrapage()`, un premier appel retourne Eva, un deuxième Bob, et un troisième lève l'exception.

## 7 Questions supplémentaires (3 points)

**7.1 (1 pt)** Pourquoi la classe `Personne` est-elle abstraite ? Que se passe-t-il à la compilation si l'on écrit `new Personne("Martin", "Alice")` ? Pourquoi `getRole()` est-elle abstraite plutôt que définie dans `Personne` ?

**7.2 (1 pt)** `NoteInvalideException` est une exception vérifiée (checked) alors que `moyenne()` lève une `IllegalStateException` non vérifiée (unchecked). Justifier ces deux choix en vous appuyant sur ce que l'appelant peut raisonnablement faire dans chaque cas.

**7.3 (1 pt)** Que permet la borne dans la déclaration `Promotion<E extends Etudiant>` ? Que se passerait-il à la compilation dans la méthode `classement()` si la classe était simplement déclarée `Promotion<E>` ?

## 8 Squelettes de classes

```
package notes;

public class NoteInvalideException extends Exception {
    public NoteInvalideException(double note) {
        super("Note invalide : " + note);
    }
}

public class EtudiantIntrouvableException extends Exception {
    public EtudiantIntrouvableException(String numero) {
        super("Aucun étudiant de numéro " + numero);
    }
}

public class FileVideException extends Exception {
    public FileVideException() {
        super("La file de rattrapage est vide");
    }
}
```

```
package notes;

public abstract class Personne {
    private String nom;
    private String prenom;
    public Personne(String nom, String prenom) {...}
    public String getNom() {...}
    public String getPrenom() {...}
    public abstract String getRole();
    public String toString() {...}
}
```

```

package notes;
import java.util.*;

public class Etudiant extends Personne {
    private final String numero;
    private List<Double> notes;
    public Etudiant(String nom, String prenom, String numero) {...}
    public String getRole() {...}
    public String getNumero() {...}
    public List<Double> getNotes() {...}
    public void ajouterNote(double note) throws NoteInvalideException {...}
    public double moyenne() {...}
    public boolean equals(Object o) {...}
    public int hashCode() {...}
    public String toString() {...}
}

```

```

package notes;

public class Enseignant extends Personne {
    private String matiere;
    public Enseignant(String nom, String prenom, String matiere) {...}
    public String getRole() {...}
    public String getMatiere() {...}
    public void noter(Etudiant e, double note) throws NoteInvalideException
    {...}
}

```

```

package notes;
import java.util.*;

public class Promotion<E extends Etudiant> {
    private List<E> etudiants;
    private LinkedList<E> rattrapage;
    public Promotion() {...}
    public boolean inscrire(E e) {...}
    public E trouver(String numero) throws EtudiantIntrouvableException {...}
    public List<E> admis() {...}
    public List<E> classement() {...}
    public String synthese() {...}
    public void constituerRattrapage() {...}
    public E convoquerSuivant() throws FileVideException {...}
}

```